ATOMICITY TEST FOR FAILED TRANSFER TRANSACTION

COBOL Programming Project | Question 12: File Testing | Difficulty: Intermediate

1. Problem Statement

A transfer should either debit the source and credit the destination, or leave both accounts unchanged. Simulate a failure that happens while a transfer is being processed. Write the logic and tests needed to prove that a failed transfer is rolled back instead of leaving only one account updated.

Input Format / Test Setup: Source balance, destination balance, amount, and a simulated failure flag.

Expected Output / Deliverable: Print COMMITTED or ROLLED BACK and final balances.

Sample Case or Evaluation Requirement: Source 1000, destination 500, amount 200, failure flag Y -> ROLLED BACK with balances unchanged.

Testing and Evaluation Requirements: Create tests for failure before debit, between debit and credit, after both updates, and normal completion. State the consistency invariant.

Implementation Assumptions:

- The input is one line: source balance, destination balance, amount, failure flag (Y or N), and an optional failure point.
- Failure point 1 means before debit, 2 means between debit and credit, and 3 means after both updates. If the flag is Y and no valid point is given, point 2 is used.
- Balances and the amount are non-negative integers of up to 9 digits.
- A transfer whose amount is greater than the source balance is also rolled back.
- The output is three lines: the status, then SOURCE and DESTINATION with their final balances.

2. Aim

To develop a COBOL program that simulates a funds transfer between two accounts with a failure that can be injected at different stages, and to prove through tests that a failed transfer is rolled back so that both accounts keep their original balances. The program prints COMMITTED or ROLLED BACK followed by the final balances.

3. Algorithm

1. Start the program.
2. Read one input line and split it into the source balance, destination balance, amount, failure flag and optional failure point.
3. Convert the balances and amount to numeric values. If the failure point is missing or not between 1 and 3, use 2.
4. Save the original source and destination balances as a before-image, and clear the abort indicator.

5. Failure point 1 (before debit): if the flag is Y and the point is 1, set the abort indicator.
6. If the transfer has not been aborted and the amount is greater than the source balance, set the abort indicator.
7. If the transfer has not been aborted, debit the source. Failure point 2 (between debit and credit): if the flag is Y and the point is 2, set the abort indicator.
8. If the transfer has not been aborted, credit the destination. Failure point 3 (after both updates): if the flag is Y and the point is 3, set the abort indicator.
9. If the abort indicator is set, restore both balances from the before-image and set the status to ROLLED BACK. Otherwise, set the status to COMMITTED.
10. Check the invariant: the total of both balances after the transfer must equal the total before it. Display INVARIANT VIOLATED if it does not.
11. Display the status, the source balance and the destination balance.
12. Stop the program.

4. Program

Note: The program works on a copy of each balance and keeps the original values in WS-OLD-SRC and WS-OLD-DST as a before-image. A failure at any point sets WS-ABORT, and the rollback paragraph then restores both balances together, so a half-completed transfer is never reported. Compile in free format: cobc -x -free transfer_atomicity.cob.

```
*> Atomic Transfer Simulation with Rollback
*> Build: cobc -x -free transfer_atomicity.cob
IDENTIFICATION DIVISION.
PROGRAM-ID. TRANSFER-ATOMICITY.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 WS-INPUT-LINE PIC X(80) VALUE SPACES.
01 WS-TOKENS.
   05 WS-TOK-1 PIC X(20) VALUE SPACES.
   05 WS-TOK-2 PIC X(20) VALUE SPACES.
   05 WS-TOK-3 PIC X(20) VALUE SPACES.
   05 WS-TOK-4 PIC X(20) VALUE SPACES.
   05 WS-TOK-5 PIC X(20) VALUE SPACES.

01 WS-SRC PIC 9(9) VALUE ZERO.
01 WS-DST PIC 9(9) VALUE ZERO.
01 WS-AMOUNT PIC 9(9) VALUE ZERO.
01 WS-OLD-SRC PIC 9(9) VALUE ZERO.
01 WS-OLD-DST PIC 9(9) VALUE ZERO.
01 WS-TOTAL-BEFORE PIC 9(10) VALUE ZERO.
01 WS-TOTAL-AFTER PIC 9(10) VALUE ZERO.

01 WS-FAIL-FLAG PIC X VALUE "N".
01 WS-FAIL-POINT PIC 9(4) VALUE 2.
01 WS-ABORT PIC X VALUE "N".
01 WS-STATUS PIC X(11) VALUE SPACES.

01 WS-DISPLAY-SRC PIC Z(8)9.
01 WS-DISPLAY-DST PIC Z(8)9.
```

```
PROCEDURE DIVISION.

MAIN-PARA.
    *> Read one line: SRC DST AMOUNT FLAG [POINT]
    ACCEPT WS-INPUT-LINE
    UNSTRING FUNCTION TRIM(WS-INPUT-LINE)
        DELIMITED BY ALL SPACE
        INTO WS-TOK-1 WS-TOK-2 WS-TOK-3
             WS-TOK-4 WS-TOK-5
    END-UNSTRING

    COMPUTE WS-SRC = FUNCTION NUMVAL(WS-TOK-1)
    COMPUTE WS-DST = FUNCTION NUMVAL(WS-TOK-2)
    COMPUTE WS-AMOUNT = FUNCTION NUMVAL(WS-TOK-3)
    MOVE FUNCTION UPPER-CASE(WS-TOK-4) TO WS-FAIL-FLAG

    IF WS-TOK-5 NOT = SPACES
        COMPUTE WS-FAIL-POINT = FUNCTION NUMVAL(WS-TOK-5)
    END-IF

    IF WS-FAIL-POINT < 1 OR WS-FAIL-POINT > 3
        MOVE 2 TO WS-FAIL-POINT
    END-IF

    PERFORM PROCESS-TRANSFER
    PERFORM CHECK-INVARIANT
    PERFORM SHOW-RESULT

    STOP RUN.

PROCESS-TRANSFER.
    *> Before-image used for rollback
    MOVE WS-SRC TO WS-OLD-SRC
    MOVE WS-DST TO WS-OLD-DST
    MOVE "N" TO WS-ABORT

    *> Failure point 1: before debit
    IF WS-FAIL-FLAG = "Y" AND WS-FAIL-POINT = 1
        MOVE "Y" TO WS-ABORT
    END-IF

    *> Business rule: insufficient funds
    IF WS-ABORT = "N" AND WS-AMOUNT > WS-SRC
        MOVE "Y" TO WS-ABORT
    END-IF

    *> Debit source; failure point 2: between debit and credit
    IF WS-ABORT = "N"
        SUBTRACT WS-AMOUNT FROM WS-SRC
        IF WS-FAIL-FLAG = "Y" AND WS-FAIL-POINT = 2
            MOVE "Y" TO WS-ABORT
        END-IF
    END-IF

    *> Credit destination; failure point 3: after both updates
    IF WS-ABORT = "N"
        ADD WS-AMOUNT TO WS-DST
        IF WS-FAIL-FLAG = "Y" AND WS-FAIL-POINT = 3
            MOVE "Y" TO WS-ABORT
        END-IF
    END-IF

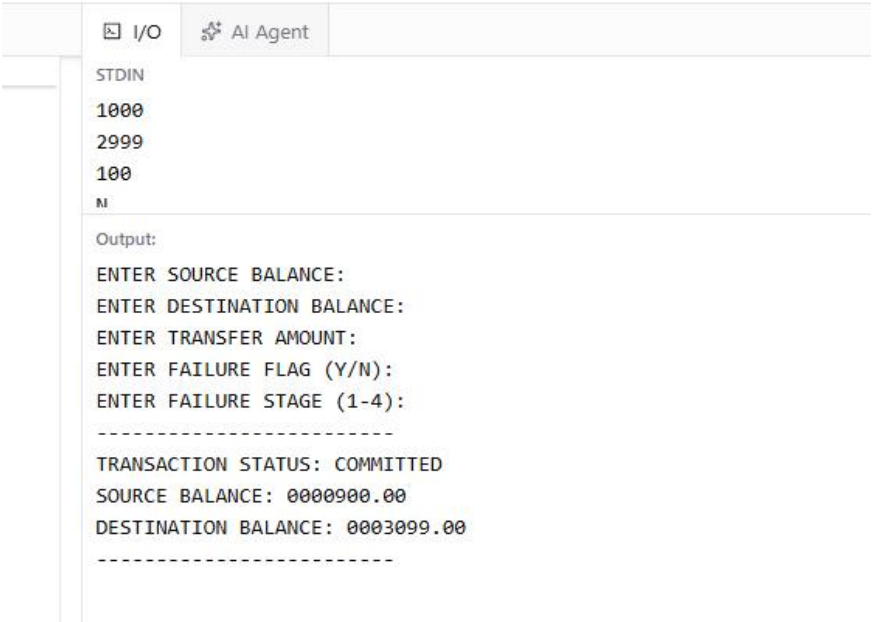
    *> Commit or roll back
```

```
    IF WS-ABORT = "Y"
        MOVE WS-OLD-SRC TO WS-SRC
        MOVE WS-OLD-DST TO WS-DST
        MOVE "ROLLED BACK" TO WS-STATUS
    ELSE
        MOVE "COMMITTED" TO WS-STATUS
    END-IF.

CHECK-INVARIANT.
    COMPUTE WS-TOTAL-BEFORE = WS-OLD-SRC + WS-OLD-DST
    COMPUTE WS-TOTAL-AFTER = WS-SRC + WS-DST
    IF WS-TOTAL-BEFORE NOT = WS-TOTAL-AFTER
        DISPLAY "INVARIANT VIOLATED"
    END-IF.

SHOW-RESULT.
    MOVE WS-SRC TO WS-DISPLAY-SRC
    MOVE WS-DST TO WS-DISPLAY-DST
    DISPLAY FUNCTION TRIM(WS-STATUS)
    DISPLAY "SOURCE " FUNCTION TRIM(WS-DISPLAY-SRC)
    DISPLAY "DESTINATION "
        FUNCTION TRIM(WS-DISPLAY-DST).
```

6. Output



7. Test Cases and Consistency Invariant

Consistency Invariant: For every transfer, the sum of the source and destination balances after processing equals the sum before processing. In addition, the final state is exactly one of two states: if the status is COMMITTED, the balances are (source − amount, destination + amount); if the status is ROLLED BACK, the balances are the original values. No state with only one account updated is ever the final state.

Each test input is stored in a text file and supplied to the program with input redirection (for example, ./transfer_atomicity < test2.txt). The expected results are listed below.

No.	Scenario	Input	Expected Output	Total Before = After
1	Failure before debit	1000 500 200 Y 1	ROLLED BACK, 1000, 500	1500 = 1500
2	Failure between debit and credit	1000 500 200 Y 2	ROLLED BACK, 1000, 500	1500 = 1500
3	Failure after both updates	1000 500 200 Y 3	ROLLED BACK, 1000, 500	1500 = 1500
4	Normal completion	1000 500 200 N	COMMITTED, 800, 700	1500 = 1500
5	Sample case (default failure point)	1000 500 200 Y	ROLLED BACK, 1000, 500	1500 = 1500
6	Insufficient funds	100 500 200 N	ROLLED BACK, 100, 500	600 = 600

Tests 2 and 3 are the key atomicity tests. Without the rollback, test 2 would leave the balances at 800 and 500, so 200 would disappear from the system, and test 3 would leave 800 and 700 even though the transfer was reported as failed. With the rollback, both tests return 1000 and 500.

8. Result

The atomic transfer simulation was implemented in COBOL. The program saves a before-image of both balances, performs the debit and the credit, and checks for a simulated failure at three points: before the debit, between the debit and the credit, and after both updates. When a failure occurs at any of these points, or when the source balance is insufficient, both balances are restored and the program prints ROLLED BACK. Otherwise it prints COMMITTED with the updated balances.

For the sample case (source 1000, destination 500, amount 200, failure flag Y), the program prints ROLLED BACK with the balances unchanged at 1000 and 500. The tests for all three failure points and for normal completion satisfy the consistency invariant, which shows that a failed transfer never leaves only one account updated.